

Regression Tree

Progetto MAP 2026

Un'applicazione Java per addestrare e visualizzare alberi di regressione. Il server gestisce i dati MySQL; il client testuale esegue le predizioni.

3

JAR: interfaccia, client e server

2

origini dati: file .dat o database

81

test JUnit superati

CONTENUTO

1	Struttura del sistema	2
2	Apprendimento	3
3	Uso essenziale	4
4	Verifica e limiti	5

01 Struttura del sistema

Il progetto ha un solo obiettivo: ottenere un valore numerico da un albero di regressione. La GUI mostra il modello; il client testuale lo usa per predire.

Due flussi



Confine funzionale. La GUI accetta sia file locali sia tabelle remote. La predizione interattiva resta nel client testuale. Senza server e database, la GUI può comunque lavorare sui file .dat.

Responsabilità

COMPONENTE	FA	NON FA
mapGUI.jar	sceglie l'origine, avvia il training, disegna l'albero, mostra statistiche e dettagli	non offre una predizione guidata all'utente
mapServer.jar	legge il database, apprende o carica un albero, salva l'archivio e serve più sessioni	non presenta un'interfaccia utente
mapClient.jar	comanda il server da terminale e guida la scelta dei rami fino alla predizione	non disegna il modello

Comunicazione remota

Il server apre un thread per client e usa stream di oggetti su TCP. I comandi principali sono: carica dati, apprendi, carica archivio e predici.

Per visualizzare un albero remoto, la GUI percorre il protocollo di predizione e ricostruisce un `VisualTree`.

Tecnologie

Java 8+ · Swing/AWT · Socket TCP · Object Streams · JDBC · MySQL Connector/J 8.0.17 · JUnit 4.12.

Codice principale: `src/data`, `src/tree`, `estensioni/gui`.

02 Apprendimento

Ogni foglia restituisce la media del target nella propria partizione. Gli split sono scelti minimizzando l'errore quadratico.

$$\hat{y}_S = (1 / |S|) \cdot \sum_{i \in S} y_i$$

$$SSE(S) = \sum_{i \in S} (y_i - \hat{y}_S)^2$$

Nel codice il campo si chiama `variance`, ma contiene la somma degli errori quadratici (SSE), non la varianza normalizzata.

1. Carica

Legge schema, attributi e target.

2. Confronta

Calcola il costo degli split validi.

3. Sceglie

Minimizza la somma delle SSE dei figli.

4. Ripete

Crea sotto-alberi o una foglia con la media.

Arresto. Una partizione diventa foglia se contiene al massimo il 10% degli esempi iniziali, calcolato con divisione intera, oppure se lo split non genera più di un ramo utile.

Dati accettati

ORIGINE	ATTRIBUTI ESPLICATIVI	TARGET
file <code>.dat</code>	discreti, dichiarati con <code>@desc</code>	numerico, dichiarato con <code>@target</code>
tabella SQL	discreti o continui, ricavati dallo schema	ultima colonna, numerica

Esempio minimo `.dat`

```
@schema 2
@desc X A,B
@desc Y A,B
@target class
@data 4
A,A,1.0
A,B,1.5
B,A,10.0
B,B,10.0
```

Classi chiave

<code>Data</code>	carica, valida e ordina gli esempi
<code>Node</code>	raccoglie stato e SSE comuni
<code>SplitNode</code>	descrive attributo e rami
<code>LeafNode</code>	conserva la media predetta
<code>RegressionTree</code>	coordina scelta e ricorsione
<code>VisualTree</code>	adatta il modello al canvas

UML: `docs/diagramma-classi.svg` e `docs/diagramma-classi-client-server.svg`.

03 Uso essenziale

L'interfaccia mostra la struttura appresa e mantiene visibili solo i comandi necessari: origine dati, training, dettagli, zoom e centratura.

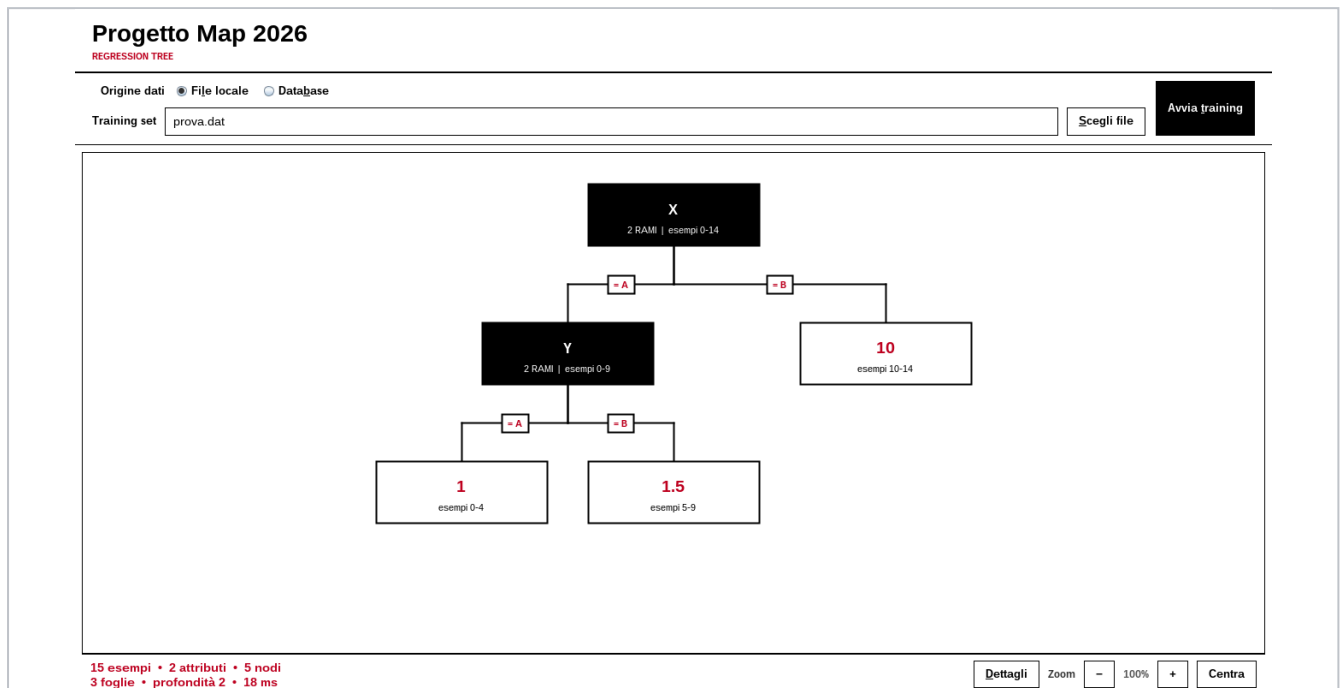


Figura 1. Albero ottenuto da prova.dat: split su X e Y, tre foglie e relative predizioni.

Nodo nero

attributo usato per lo split

Etichetta sul ramo

condizione da seguire

Valore rosso

predizione della foglia

GUI

```
java -jar distribution/gui/mapGUI.jar
```

1. Scegliere **File locale** o **Database**.
2. Indicare file oppure server e tabella.
3. Premere **Avvia training**.

La modalità locale non richiede rete né database.

SERVER E CLIENT

```
mysql -u root -p \  
<  
distribution/server/sql_file/map6_setup.sql  
cd distribution/server  
java -jar mapServer.jar 8080  
  
# da un secondo terminale  
java -jar distribution/client/mapClient.jar \  
localhost 8080
```

Attenzione. Lo script SQL ricrea il database MapDB. Il Connector/J deve restare accanto a mapServer.jar. Il server usa per impostazione corrente localhost:3306, utente MapUser.

04 Verifica e limiti

La baseline corrente è stata compilata e verificata su OpenJDK 26.0.2. La compilazione usa il livello Java 8.

81

test JUnit

13

classi di test

0

fallimenti

0

errori

AREA	CASI	COSA VERIFICA
data	17	schema, parsing, valori, ordinamento e input errati
tree	26	SSE, split, foglie, ricorsione, regole e predizione
utility + MainTest	19	lettura da console, limiti, EOF e flussi principali
estensioni.gui	19	adapter, canvas, zoom, training e stati della GUI

Due controlli eseguibili aggiuntivi verificano le statistiche visuali e il protocollo database con un server simulato.

```
$ java ... org.junit.runner.JUnitCore [13 classi]
JUnit version 4.12
.....
Time: 0.996

OK (81 tests)
```

Limiti principali

Integrazione

Mancano test automatici con database reale, socket end-to-end e più client concorrenti.

Configurazione e rete

Credenziali e database sono fissi; il protocollo non usa TLS né autenticazione.

GUI sincrona

Training e rete lavorano sull'EDT. Dataset o alberi grandi possono bloccare la finestra.

Distribuzione

Gli archivi usano la serializzazione Java e il sorgente del server non è incluso nel repository.

Valutazione. Il progetto realizza il flusso completo: acquisizione dei dati, apprendimento, visualizzazione e predizione. Le priorità successive sono test end-to-end, configurazione esterna e lavoro asincrono nella GUI.

Documenti collegati

Manuale utente: docs/manuale_utente.pdf

Diagrammi UML: docs/diagramma-classi.pdf · docs/diagramma-classi-client-server.pdf

Non è dichiarata una percentuale di coverage: non è stata misurata.